

Smart Scheduler

Indice

1. Introduzione
 2. Definizione del problema
 3. Architettura generale e flusso di esecuzione
 4. Strutture dati e comunicazione tra gli agenti
 5. Integrazione con il modello linguistico
 6. Tecniche di prompting e pattern multi-agente
 7. Implementazione dettagliata degli agenti
 8. Produzione degli output e analisi dei risultati
 9. Conclusioni
-

1. Introduzione

Il presente progetto illustra la progettazione e lo sviluppo di un sistema multi-agente nato per automatizzare e ottimizzare la pianificazione dei turni all'interno di un contesto ospedaliero.

Il problema affrontato non consiste soltanto nel distribuire un certo numero di turni tra i dipendenti, ma richiede di conciliare due esigenze differenti:

- rispettare rigorosamente i requisiti organizzativi e normativi, indicati come **vincoli hard**;
- tenere conto, per quanto possibile, delle richieste individuali dei lavoratori, trattate come **preferenze soft**.

Tra i vincoli obbligatori rientrano, per esempio, la copertura minima dei turni, il limite massimo di ore settimanali, il riposo successivo a un turno notturno e il carico mensile richiesto. Le preferenze soft comprendono invece turni graditi o sgraditi, richieste su date specifiche, giorni della settimana preferiti, riposi desiderati e disponibilità dichiarate dai singoli dipendenti.

Il progetto adotta un'architettura **neuro-simbolica**:

- i **Large Language Models** interpretano testi, producono strutture dati, generano una strategia e propongono le turnazioni;
- **LangGraph** orchestra i nodi del workflow e gestisce i cicli di correzione;
- **Pydantic** impone schemi tipizzati agli output degli agenti;
- **OR-Tools CP-SAT** verifica matematicamente i vincoli obbligatori;
- una funzione deterministica calcola la fairness e individua il lavoratore maggiormente penalizzato.

L'obiettivo non è quindi affidare l'intera soluzione a un singolo modello generativo. Il sistema distribuisce il problema tra componenti specializzate, applica controlli incrociati e conserva l'ultimo piano valido quando una nuova proposta non supera le verifiche.

Nelle sezioni successive vengono descritte le scelte progettuali e vengono analizzati i risultati ottenuti nei due scenari applicativi previsti: il Caso d'Uso A, con lavoratori omogenei, e il Caso d'Uso B, con lavoratori standard e specializzati

2. Definizione del problema

2.1 Orizzonte temporale

La pianificazione copre il periodo compreso tra il **7 dicembre 2026** e il **6 gennaio 2027**, per un totale di **31 giorni**.

Ogni giornata è suddivisa in tre fasce:

Codice	Turno	Orario	Ore	Unità di carico
M	Mattina	08:00-14:00	6	1
P	Pomeriggio	14:00-20:00	6	1
N	Notte	20:00-08:00	12	2
R	Riposo	-	0	0

Il turno di notte contribuisce con due unità al carico mensile, coerentemente con la sua durata doppia rispetto ai turni di mattina e pomeriggio.

2.2 Scenari applicativi

Il progetto considera due configurazioni:

Caso A - lavoratori omogenei

Tutti i lavoratori sono considerati interscambiabili. Ogni turno deve essere coperto da almeno **due dipendenti**.

Il file di output disponibile comprende **13 lavoratori**, identificati dalle lettere da **A** a **M**.

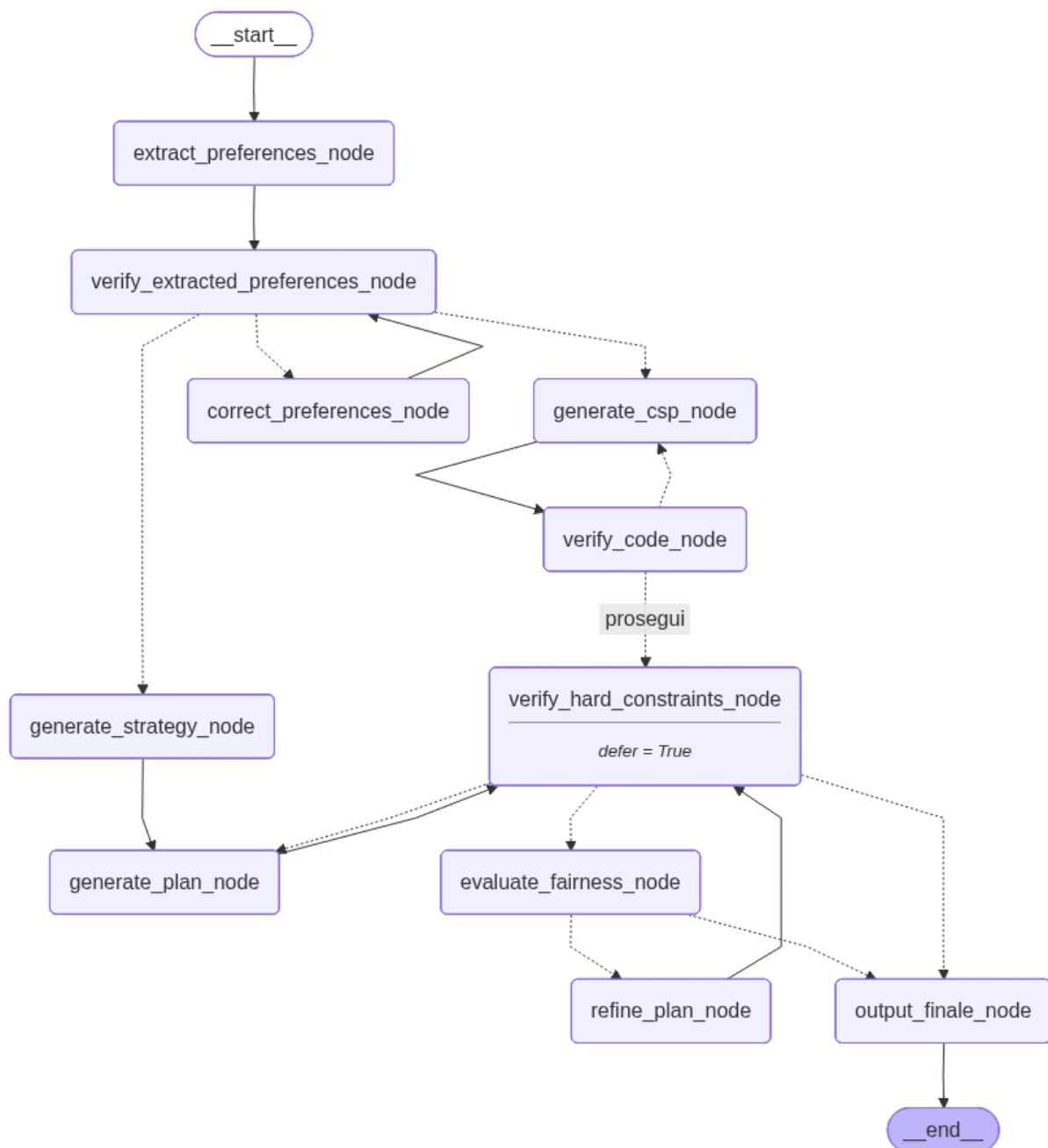
Caso B - lavoratori standard e specializzati

Il personale è suddiviso in:

- **13 lavoratori standard**, da **A** a **M**;
 - **6 lavoratori specializzati**, da **N** a **S**.
-

3. Architettura generale e flusso di esecuzione

Il workflow è definito in `main.py` attraverso un `StateGraph` di LangGraph. Ogni nodo riceve lo stato condiviso, svolge un compito specifico e restituisce soltanto i campi aggiornati.



Flusso del sistema multi-agente implementato con LangGraph

Fase 1 - estrazione e validazione delle preferenze

Il processo parte dal nodo di ingresso e gestisce il testo delle preferenze

1. `extract_preferences_node` interpreta il testo in linguaggio naturale, individua preferenze, indisponibilità, richieste su date specifiche e ruolo del dipendente, restituendo un oggetto Pydantic strutturato.

2. `verify_extracted_preferences_node` confronta il testo originale con l'output estratto e verifica completezza, coerenza semantica e correttezza della struttura.
3. `correct_preferences_node` riceve il feedback del verificatore e corregge le sole preferenze segnalate, quindi rimanda il risultato alla verifica. Il loop continua finché l'estrazione non è considerata valida.
4. Il risultato corretto viene nuovamente verificato.

Fase 2-Preparazione parallela del modello e della strategia

Dopo la validazione delle preferenze il grafo avvia due rami complementari:

- il ramo che genera e verifica il codice CP-SAT;
- il ramo strategico che produce le istruzioni operative e genera la prima bozza del piano.

Nel primo ramo:

`generate_csp_node -> verify_code_node`

`generate_csp_node` traduce i vincoli hard espressi in linguaggio naturale in una funzione

Python che costruisce un modello CP-SAT. In parallelo genera anche una funzione di

feedback procedurale, usata per spiegare le violazioni rilevate nei piani.

`verify_code_node` agisce come critico del codice generato, confrontando ogni regola del testo

originale con il CSP e restituendo errori strutturati. In presenza di anomalie il flusso torna al

Coding Agent.

Nel secondo ramo:

`generate_strategy_node -> generate_plan_node`

`generate_strategy_node` produce una strategia di pianificazione testuale con ordine di assegnazione, protocollo di correzione e checklist di autocontrollo. La strategia viene poi

iniettata nel prompt del generatore del piano.

I due rami convergono nel nodo differito `verify_hard_constraints_node`. La scelta

`defer=True` consente di posticipare il controllo fino a quando gli aggiornamenti

necessari allo stato sono disponibili.

Fase 3 - generazione della bozza

`generate_plan_node` Il generatore produce una matrice completa composta da 31 valori per ogni dipendente. I simboli ammessi sono esclusivamente `M`, `P`, `N` e `R`.

Se una bozza precedente è risultata non valida, il generatore riceve:

- il piano precedente;
- l'elenco delle violazioni;
- la strategia prodotta dal Planning Agent.

L'obiettivo è correggere localmente gli errori anziché ricominciare senza utilizzare il feedback già ottenuto.

Fase 4 - verifica hard e fairness

Il nodo `verify_hard_constraints_node` verifica il piano attraverso CP-SAT.

- Se il primo piano è infeasible, il flusso ritorna al generatore.
- Se il piano è valido, viene calcolata la fairness.
- Se una proposta di raffinamento è infeasible, il sistema interrompe il ciclo e conserva il precedente `best_plan`.

Il nodo `evaluate_fairness_node` confronta il piano corrente con il migliore piano già accettato. La priorità è ridurre la penalità massima; in caso di parità viene confrontata la media.

Fase 5 - raffinamento

`refine_plan_node` prova a migliorare la situazione del lavoratore più penalizzato attraverso scambi tra dipendenti compatibili.

La proposta raffinata viene sempre sottoposta di nuovo a:

1. verifica dei vincoli hard;
2. ricalcolo della fairness;
3. confronto con il piano migliore.

Ogni proposta raffinata viene nuovamente verificata da OR-Tools

Terminazione

Il nodo `output_finale_node` esporta l'ultimo `best_plan` genera la tabella Excel e salva un audit log JSON contenente contatori di iterazione, punteggi di fairness, dipendente

4. Strutture dati e comunicazione tra gli agenti

4.1 Stato condiviso

Il passaggio di informazioni tra agenti e risolutori avviene tramite un unico oggetto di stato

condiviso. La classe SchedulerForm, definita con Pydantic denominato

`SchedulerForm`, funge da memoria globale del workflow e contiene sia gli input iniziali sia i risultati intermedi.

```
class SchedulerForm(BaseModel):
    input: dict[str, str]
    errori_codice: ListaErroriCodice = None
    vincoli_soft: VincoliStrutturati = None
    preferenze_valide: PreferenzeValidare = None
    n_iter_correzioni: int = 0
    planner_strategy: str = None
    best_plan: Piano = None
    piano_attuale: Piano = None
    n_iter_piano: int = 0
    hard_constraints_valid: bool = None
    feedback_errori_hard: str = None
    n_iter_raffinazioni: int = 0
    dipendente_piu_sfortunato: list[str] = []
    fairness_score: dict[str, float] = None
    condizione_di_stop: str = None
    terminazione_raggiunta: bool = None
    condizione_di_stop: str = None
    terminazione_raggiunta: bool = None
```

I campi principali sono:

Campo	Funzione
<code>input</code>	Contiene il testo delle preferenze e dei vincoli hard.
<code>vincoli_soft</code>	Memorizza le preferenze strutturate.
<code>preferenze_valide</code>	Contiene il giudizio del verificatore e i suggerimenti.

Campo	Funzione
<code>errori_codice</code>	Contiene gli errori rilevati nel CSP generato.
<code>planner_strategy</code>	Conserva la strategia prodotta dal Planning Agent.
<code>piano_attuale</code>	Rappresenta la turnazione attualmente in esame.
<code>best_plan</code>	Conserva l'ultimo piano valido e accettato dal criterio di fairness.
<code>feedback_errori_hard</code>	Descrive le violazioni riscontrate nella bozza.
<code>fairness_score</code>	Associa a ogni dipendente la propria penalità.
<code>dipendente_piu_sfortunato</code>	Mantiene la cronologia dei lavoratori con penalità massima.
<code>condizione_di_stop</code>	Specifica il motivo della terminazione.
<code>terminazione_raggiunta</code>	Determina il passaggio al nodo finale.

I campi `errori_codice` e `piano_attuale` utilizzano il reducer `last_value`. Questo aspetto è importante perché il workflow contiene rami paralleli: quando arrivano più aggiornamenti, viene mantenuto l'ultimo valore non nullo.

4.2 Preferenze strutturate

`VincoliStrutturati` contiene una lista di oggetti `PreferenzeDipendente`. Ogni oggetto può includere:

- identificativo del dipendente;
- indicazione del ruolo specializzato;
- turni desiderati e da evitare;
- giorni graditi e sgraditi;
- richieste su date specifiche;
- limite di emergenze;
- riposo preferito;
- tipologie di turno consecutivo poco tollerate.

Ad ogni preferenza viene associata un livello di priorità specificata nell'enumerazione:

```
class ImportanzaPreferenza(str, Enum):
    LIEVE = "LIEVE"
    MODERATA = "MODERATA"
```

```
ALTA = "ALTA"
VITALE = "VITALE"
```

La tipizzazione consente di bloccare valori non ammessi e di intercettare output incompleti prima che entrino nelle fasi successive.

4.3 Rappresentazione del piano

Il piano è definito mediante due classi:

- `TurniDipendente`, che associa un lavoratore a una lista di turni;
- `Piano`, che contiene tutti gli assegnamenti.

La rappresentazione per dipendente facilita i controlli orizzontali, come il calcolo delle ore o del carico. Il metodo `to_dict()` permette invece di ottenere una struttura del tipo:

```
{
    "A": ["M", "R", "N", ...],
    "B": ["P", "M", "R", ...]
}
```

L'uso di questa struttura permette al modello linguistico di leggere una riga per dipendente per volta, permettendo anche ai *thinking tokens* dell'LLM di contare e validare facilmente il carico di lavoro.

5. Integrazione con il modello linguistico

5.1 Provider effettivamente utilizzato

La versione analizzata utilizza `ChatGoogleGenerativeAI` e quindi è collegata a **Google Gemini**. I nomi dei modelli vengono letti dalle variabili d'ambiente e possono essere differenziati per i

compiti di generazione del piano e di produzione del codice. Questa scelta rende il sistema più

facilmente aggiornabile e consente di sostituire il modello senza modificare la logica del workflow.

5.2 Configurazione degli agenti

Componente	Temperatura	Thinking level	Output strutturato	Modello LLM
Estrazione preferenze	0.1	predefinito	VincoliStrutturati	Gemini 2.5 Flash
Verifica preferenze	0.3	predefinito	PreferenzeValidate	Gemini 2.5 Flash
Correzione preferenze	0.1	predefinito	VincoliStrutturati	Gemini 2.5 Flash
Generazione CSP	0.0	high	codice testuale	Gemini 3.5 Flash
Generazione feedback hard	0.0	high	codice testuale	Gemini 3.5 Flash
Verifica CSP	0.3	predefinito	ListaErroriCodice	Gemini 2.5 Flash
Planning Agent	0.3	predefinito	testo Markdown	Gemini 2.5 Flash
Generazione piano	0.0	high	Piano	Gemini 3.5 Flash
Raffinamento	0.0	high	Piano	Gemini 3.5 Flash

Le temperature basse sono appropriate per i compiti nei quali la variazione creativa sarebbe dannosa. Una temperatura leggermente superiore viene invece usata nei nodi critici e strategici, che devono formulare analisi e suggerimenti.

Per la generazione (e raffinazione) delle turnazioni e del codice la scelta è ricaduta sul modello `gemini-3.5-flash`. Questa decisione è stata dettata dalla stringente necessità di dotare il sistema di capacità avanzate di ragionamento, fondamentali nel processare i vincoli orizzontali dei dipendenti e nel pianificare operazioni complesse di scambio turni.

Sebbene sul mercato esistano modelli dotati di abilità deduttive di altissima fascia (come **GPT-5.5** o lo stesso **Gemini 3.1 Pro**), l'utilizzo di questi comporta costi API significativamente più elevati rispetto al modello **3.5-Flash** il cui utilizzo è disponibile anche gratuitamente (seppur in modo limitato).

Dalla Figura 2, si può notare come il modello usato riesce ad ottenere prestazioni simili ai modelli migliori sui task di ragionamento, garantendo inoltre prestazioni anche più elevate sui task agentici.

Benchmark			Gemini 3.5 Flash	Gemini 3 Flash	Gemini 3.1 Pro	Claude Sonnet 4.6	Claude Opus 4.7	GPT-5.5
Coding	Terminal-bench 2.1 Agentic terminal coding	Terminus-2 harness	76.2%	58.0%	70.3%	-	66.1%	78.2%
	SWE-Bench Pro (Public) Diverse agentic coding tasks	Single attempt	55.1%	49.6%	54.2%	-	64.3%	58.6%
Agentic	MCP Atlas Multi-step workflows using MCP		83.6%	62.0%	78.2%	69.5%	79.1%	75.3%
	Toolathlon Real-world general tool use		56.5%	49.4%	-	-	-	55.6%
UI control	OSWorld-Verified Agentic computer use		78.4%	65.1%	76.2%	72.5%	78.0%	78.7%
Expert tasks	Finance Agent v2 Financial analysis and decision-making		57.9%	42.6%	43.0%	51.0%	51.5%	51.8%
	GDPval-AA Economically valuable knowledge work	Elo	1656	1204	1314	1676	1753	1769
Multimodal	CharXiv Reasoning Information synthesis from complex charts	No tools	84.2%	80.3%	83.3%	72.4%	82.1%	84.1%
	MMMU-Pro Multimodal understanding and reasoning	No tools	83.6%	81.2%	80.5%	74.5%	75.2%	81.2%
	Blueprint-Bench 2 Agentic spatial reasoning	Normalized score	33.6%	0.0%	26.5%	6.7%	24.5%	36.2%
Long context	MRCR v2 (8-needle) Long context performance	128k (average) 1M (pointwise)	77.3% 26.6%	67.2% 22.1%	84.9% 26.3%	84.9% -	59.3% -	94.8% -
	Humanity's Last Exam Academic reasoning (full set, text + MM)		40.2%	33.7%	44.4%	33.2%	46.9%	41.4%
Reasoning	ARC-AGI-2 Abstract reasoning puzzles		72.1%	33.6%	77.1%	58.3%	75.8%	84.6%

Figura 2: Confronto tra Gemini 3.5 Flash ed altri modelli con ragionamento.

6. Implementazione dettagliata degli agenti e pattern utilizzati

Questa sezione descrive il ruolo dei diversi agenti, le informazioni che ricevono, il tipo di elaborazione che svolgono e i pattern di prompting applicati. L'architettura è costruita in modo che nessun agente controlli autonomamente l'intero processo. Ogni nodo svolge un compito circoscritto e il suo risultato viene utilizzato, verificato o corretto dalle componenti successive.

Il coordinamento avviene tramite lo stato condiviso `SchedulerForm`. Gli agenti non comunicano attraverso conversazioni indipendenti, ma leggono e aggiornano campi comuni. LangGraph può così mantenere memoria delle preferenze estratte, del codice generato, delle bozze precedenti, degli errori riscontrati e del miglior piano valido individuato fino a quel momento.

6.1 Agente di estrazione delle preferenze

L'agente di estrazione rappresenta il primo collegamento tra il linguaggio naturale e la struttura formale del sistema. Il suo compito consiste nel leggere il testo contenente le richieste dei dipendenti e trasformarlo in un insieme di preferenze tipizzate.

Il nodo riceve il testo presente in `state.input["preferences"]` e analizza separatamente le informazioni relative a ciascun lavoratore. Durante questa fase riconosce i turni desiderati, quelli da evitare, i giorni della settimana graditi o sgraditi, le richieste

associate a date specifiche, il giorno di riposo preferito, l'eventuale ruolo specializzato e le sequenze di turni considerate poco tollerabili.

L'agente deve anche interpretare l'intensità linguistica della richiesta. Una frase come "preferirei non lavorare di notte" viene trattata in modo diverso da "non posso assolutamente lavorare di notte". Nel primo caso la preferenza può essere classificata come moderata, mentre nel secondo il peso associato sarà più elevato. Questa distinzione è importante perché la funzione di fairness utilizza tali pesi per calcolare il livello di penalizzazione di ogni lavoratore.

Dal punto di vista del prompting, questo agente applica innanzitutto il **Role Prompting**, poiché il modello viene istruito ad agire esclusivamente come estrattore di informazioni e non come generatore della turnazione. Utilizza inoltre la **Task Decomposition**, perché il testo viene scomposto in categorie indipendenti: turni, giorni, date, riposi, qualifiche e livelli di importanza. Questa scomposizione riduce il rischio che il modello si concentri soltanto sulle preferenze più evidenti e trascuri informazioni temporali o numeriche.

Nell'estrattore è presente il **Few-Shot Prompting**, il prompt contiene esempi che mostrano come convertire frasi narrative in campi strutturati, come distinguere una richiesta positiva da una negativa e come assegnare un livello di importanza.

Viene infine applicata la **Context Injection**, perché l'agente riceve il testo delle preferenze e lo schema necessario alla sua trasformazione, senza essere sovraccaricato con il piano, il codice CP-SAT o i risultati delle fasi successive. Lo structured output completa questa strategia: il modello non restituisce testo libero, ma un oggetto `VincoliStrutturati`, che Pydantic può validare automaticamente.

6.2 Agente di verifica delle preferenze

Il verificatore delle preferenze svolge il ruolo di revisore esterno, costituendo la componente critica del pattern **Critic-Corrector**.

La scelta di usare un secondo agente evita che il modello responsabile dell'estrazione debba anche giudicare il proprio lavoro.

Il verificatore riceve contemporaneamente il testo originale e l'oggetto prodotto dall'estrattore. Il suo compito consiste nel controllare che ogni informazione importante sia stata interpretata correttamente. La revisione riguarda soprattutto omissioni, campi inventati, errori nelle date, assegnazioni errate del ruolo e livelli di importanza non coerenti con la formulazione usata dal dipendente.

Il pattern principale applicato è il **Role Prompting**. Il modello viene definito come critico e revisore, questa distinzione limita la possibilità che riscriva liberamente informazioni già corrette.

Anche in questo nodo viene usata la **Context Injection**. Il modello riceve soltanto il testo originale, l'estrazione da controllare e la descrizione dello schema.

Il risultato è un oggetto `PreferenzeValidate`, composto da un indicatore di validità e da una lista di suggerimenti. Quando `valida` è impostato a `True`, il flusso può procedere. In caso contrario, il verificatore specifica per quale dipendente è necessaria una correzione e descrive la modifica richiesta.

6.3 Agente di correzione delle preferenze

L'agente di correzione interviene quando il verificatore ha rilevato almeno un errore. Riceve il testo originale, le preferenze precedentemente estratte e i suggerimenti associati ai dipendenti coinvolti.

La sua funzione consiste nell'applicare modifiche mirate. Le preferenze già considerate corrette devono essere conservate, mentre quelle segnalate vengono ricostruite tenendo conto delle osservazioni del verificatore. La temperatura molto bassa favorisce un comportamento conservativo e riduce il rischio di introdurre nuove interpretazioni non richieste.

Questo agente completa il ciclo **Critic-Corrector** avviato dal verificatore, la correzione non conclude immediatamente la fase: il nuovo risultato viene inviato nuovamente al verificatore, che controlla se gli errori siano stati effettivamente risolti.

Il nodo applica anche il pattern di **Self-Correction**, inteso qui a livello di workflow. Il sistema utilizza l'errore prodotto da una fase precedente per migliorare l'output senza ripartire completamente da zero. Non si tratta di una semplice autocorrezione dello stesso prompt, ma di una correzione guidata da un agente esterno.

La **Context Injection** consente di fornire al correttore soltanto gli elementi utili: testo originale, preferenze estratte e suggerimenti. Lo structured output assicura che il nuovo risultato sia ancora un oggetto di tipo `VincoliStrutturati`.

Dopo la correzione, il campo `state.vincoli_soft` viene aggiornato e il flusso ritorna al verificatore. Il workflow procede soltanto quando l'intera struttura viene considerata coerente con il testo originale.

6.4 Coding Agent per la generazione del CSP

Il Coding Agent traduce i vincoli hard espressi in linguaggio naturale in un modello CP-SAT eseguibile. Si tratta di uno dei passaggi più delicati, perché il modello deve trasformare una descrizione narrativa in una formulazione simbolica precisa.

L'agente genera una funzione Python con firma prestabilita:

```
def crea_modello_vincoli_hard(model, shifts, std_nurses, spec_nurses):
```

La funzione crea le variabili booleane `shifts[(n, d, s)]` e aggiunge al modello le condizioni richieste. Ogni variabile indica se un lavoratore è assegnato a uno specifico turno in un determinato giorno.

$$shift[(n, d, s)] = \begin{cases} 1 & \text{se il Dipendente } n \text{ è assegnato al turno } s \text{ nel giorno } d \\ 0 & \text{altrimenti} \end{cases}$$

Il **Role Prompting** definisce il modello come programmatore specializzato in OR-Tools. Il suo compito non è proporre una turnazione, ma tradurre i requisiti in codice. Questa delimitazione separa nettamente la generazione del modello dalla generazione del piano.

La **Task Decomposition** viene applicata trasformando ciascuna regola della traccia in un blocco logico distinto. Il massimo di un turno al giorno, la copertura, il riposo post-notte, il carico mensile e il limite settimanale vengono affrontati separatamente. In questo modo il codice può essere confrontato con i requisiti uno alla volta.

Il prompt richiede inoltre commenti prima di ogni vincolo, nei quali devono essere esplicitati obiettivo, indici e primitiva OR-Tools utilizzata. Questo comportamento rappresenta una forma di ragionamento guidato e rende più leggibile la corrispondenza tra requisito e implementazione, facilitando la correzione al Code Verifier.

Quando il Code Verifier segnala un errore, il Coding Agent riceve il codice precedente e il feedback strutturato. In questa fase applica **Context Injection** e **Self-Correction**: non rigenera il modello alla cieca, ma modifica una soluzione già esistente utilizzando informazioni precise sugli errori.

Il Coding Agent partecipa inoltre al secondo ciclo **Critic-Corrector** del sistema. Esso svolge il ruolo di generatore, mentre il Code Verifier agisce come critico. Il nuovo codice viene sottoposto nuovamente a revisione fino a quando non vengono più segnalate anomalie.

6.5 Generatore del feedback sui vincoli hard

All'interno dello stesso modulo viene generata anche una funzione dedicata alla spiegazione delle violazioni:

```
def estrai_feedback_errori_hard(
    piano_assegnamenti,
    std_nurses,
    spec_nurses
):
```

Questa funzione ha una finalità diversa rispetto al modello CP-SAT. Il solver può dichiarare un piano infeasible, ma tale stato non sempre fornisce indicazioni sufficienti per guidare il modello linguistico nella correzione.

Il generatore del feedback crea controlli procedurali in Python. Quando trova una violazione, prova a indicare il dipendente interessato, il giorno, il tipo di vincolo e il valore non conforme.

Anche qui viene usata la **Task Decomposition**, perché ogni vincolo hard viene trasformato in un controllo procedurale specifico. La funzione può così distinguere una copertura insufficiente da un eccesso di ore o da una sequenza notturna non valida.

La funzione di feedback partecipa soprattutto al ciclo di **verifica neuro-simbolica**. CP-SAT stabilisce matematicamente che il piano non è ammissibile; il controllo procedurale converte questa informazione in un messaggio utilizzabile dall'LLM. Il risultato viene poi reinserito nel generatore del piano attraverso la **Context Injection**.

6.6 Code Verifier Agent

Il Code Verifier controlla il codice generato dal Coding Agent, il suo obiettivo è verificare che esista una corrispondenza tra il testo dei vincoli hard e la loro implementazione in OR-Tools.

Il verificatore esamina commenti e istruzioni, controlla se tutte le regole sono state rappresentate, se i cicli attraversano correttamente giorni e lavoratori, se le finestre settimanali sono coerenti e se le implicazioni logiche utilizzano correttamente le primitive CP-SAT.

Il **Role Prompting** assegna al modello il ruolo esclusivo di revisore tecnico, esso non deve sostituire liberamente il codice, ma individuare errori e proporre interventi puntuali.

Il nodo rappresenta il critico del secondo ciclo **Critic-Corrector**. Il Coding Agent genera il modello, il Code Verifier lo confronta con la fonte originale e, in presenza di problemi, restituisce una struttura contenente il vincolo interessato, la categoria dell'errore, una descrizione e un suggerimento.

La **Context Injection** limita il prompt al testo dei vincoli, al codice prodotto e ai commenti di ragionamento. Il verificatore non necessita della matrice finale o dei punteggi di fairness.

Se la lista degli errori è vuota, il flusso può procedere. In caso contrario, LangGraph riporta l'esecuzione al Coding Agent. Il Code Verifier rimane comunque un agente LLM: la sua revisione semantica è utile, ma non equivale all'esecuzione reale del codice, che avviene successivamente nel validatore simbolico.

6.7 Planning Agent

Il Planning Agent non produce direttamente la matrice dei turni ma il suo compito consiste nel formulare una strategia destinata al generatore.

L'output è suddiviso in una strategia di generazione, un protocollo di correzione nel caso di iterazioni ripetute e una sezione di autocontrollo.

La necessità di questo agente è dovuta al fatto che i vincoli hard sono memorizzati in un file di input, pertanto per ottenere una strategia dinamica in linguaggio naturale è necessario usare un agente che la generi.

6.8 Agente di generazione del piano

L'agente di generazione costruisce la prima turnazione completa usando il calendario, vincoli hard, preferenze strutturate e strategia prodotta dal Planning Agent.

Il **Role Prompting** assegna al modello il compito specifico di generare la matrice, distinguendolo sia dal planner sia dal validatore. Il prompt stabilisce inoltre una gerarchia precisa: i vincoli hard hanno priorità, mentre le preferenze soft devono essere soddisfatte soltanto quando non compromettono la fattibilità.

La **Context Injection**: alla prima iterazione il generatore riceve calendario, strategia, vincoli e preferenze e nei tentativi successivi vengono aggiunti il piano precedente e il feedback sulle violazioni. Il contesto cambia quindi dinamicamente in base allo stato del workflow.

La checklist prodotta dal Planning Agent consente di applicare il **Self-Control** prima della risposta, in questo modo il modello viene invitato a ricontrollare copertura, carichi, ore e sequenze.

Quando il validatore respinge la bozza, il generatore applica la **Self-Correction** utilizzando il piano precedente e il feedback per intervenire sui punti problematici.

L'output è un oggetto `Piano`, formato da un insieme di `TurniDipendente`. Lo structured output garantisce che ogni lavoratore possieda una lista di valori compatibili con lo schema, ma non dimostra il rispetto dei vincoli globali.

6.9 Validatore simbolico dei vincoli hard

Il nodo `verify_hard_constraints_node` è un agente simbolico e deterministico basato su OR-Tools.

Il validatore importa dinamicamente il modello CP-SAT e la funzione di feedback generati dal Coding Agent, successivamente crea un nuovo `CpModel`, costruisce le variabili e collega la matrice proposta alle variabili decisionali.

Questo nodo realizza il nucleo della **verifica neuro-simbolica**: la componente LLM produce una proposta in linguaggio strutturato, mentre la componente simbolica controlla matematicamente la sua ammissibilità.

Se il solver restituisce `FEASIBLE` oppure `OPTIMAL`, il piano può passare alla valutazione della fairness, altrimenti il sistema genera un feedback e ritorna al generatore.

Durante il raffinamento, se esiste già un `best_plan`, una proposta non valida non sostituisce il piano precedente. Il workflow termina conservando l'ultima soluzione ammissibile.

6.10 Agente di valutazione della fairness

La valutazione della fairness è anche in questo caso una componente deterministica, non un agente LLM.

Per ogni dipendente viene calcolato un punteggio di penalità. Le preferenze non rispettate aumentano il valore, mentre alcune preferenze soddisfatte possono ridurlo. Un valore elevato rappresenta una condizione peggiore.

L'agente adotta una funzione di costo basata sulla classificazione delle priorità avvenuta nelle fasi iniziali:

Importanza	Valore
<code>LIEVE</code>	2
<code>MODERATA</code>	5
<code>ALTA</code>	8
<code>VITALE</code>	10

Una richiesta classificata come `VITALE` produce quindi un impatto più rilevante rispetto a una richiesta `LIEVE`.

Dopo aver calcolato i punteggi, il nodo individua il lavoratore con penalità massima. Questo dipendente viene considerato il più svantaggiato e il suo identificativo viene fornito al raffinatore mediante **Context Injection**.

Il confronto tra piani segue un criterio lessicografico. Prima viene confrontata la penalità massima; soltanto in caso di parità viene confrontata la media. Quando il

nuovo piano migliora il criterio, viene salvato in `state.best_plan`. Quando invece peggiora o non produce vantaggi, il workflow registra una condizione di stop.

6.11 Agente raffinatore

Il raffinatore riceve un piano già valido ed il suo obiettivo non è correggere una violazione hard, ma migliorare la distribuzione delle preferenze.

Il **Role Prompting** definisce il modello come agente di ricerca locale, esso non deve costruire un nuovo piano completamente indipendente, ma proporre modifiche circoscritte a una soluzione già ammissibile.

Attraverso la **Context Injection**, il nodo riceve il piano corrente, le preferenze, i vincoli hard e l'identificativo del lavoratore più penalizzato. Questa informazione concentra la ricerca sul dipendente che determina il valore massimo della fairness.

Il raffinatore applica una forma di **Self-Correction**, poiché tenta di migliorare una soluzione precedente sulla base di una valutazione esterna. Il prompt limita la ricerca a scambi uno-a-uno tra lavoratori della stessa qualifica, riducendo il rischio di compromettere la copertura degli specializzati.

Anche questo agente partecipa al ciclo di **verifica neuro-simbolica**. La proposta raffinata non viene mai accettata direttamente: passa nuovamente da CP-SAT e dalla funzione di fairness.

Il raffinatore deve inoltre valutare gli effetti dello scambio sugli altri lavoratori. Un miglioramento ottenuto rendendo un collega ancora più penalizzato non costituisce un progresso secondo il criterio adottato.

6.12 Nodo di output finale

Il nodo `output_finale_node` conclude il workflow, la sua funzione è rendere persistenti i risultati prodotti e verificati dalle fasi precedenti. Il nodo esporta `state.best_plan`, cioè l'ultimo piano che ha superato la verifica hard e il confronto di fairness.

Il file Excel organizza i giorni sulle colonne e i turni sulle righe. In ogni cella vengono inseriti gli identificativi dei dipendenti assegnati.

Il nodo produce anche un audit log JSON, nel quale vengono registrati il numero di iterazioni, i punteggi di fairness, il lavoratore più penalizzato e la condizione di arresto.

Questo nodo completa il principio di verificabilità dell'architettura: i pattern di prompting guidano la generazione e la correzione, mentre l'output finale conserva soltanto un risultato già controllato dalle componenti deterministiche. L'audit log permette inoltre di ricostruire il percorso seguito dal workflow e il motivo della terminazione.

7. Produzione degli output e analisi dei risultati

Il nodo finale produce due artefatti principali: una tabella Excel contenente la turnazione e un file JSON utilizzato come audit log.

Il foglio Excel rappresenta il risultato operativo del sistema. I giorni sono organizzati sulle colonne, mentre i turni di mattina, pomeriggio e notte sono riportati sulle righe. In ciascuna cella vengono inseriti gli identificativi dei lavoratori assegnati.

Il riposo non viene riportato esplicitamente nella tabella, poiché può essere dedotto dall'assenza del lavoratore nei tre turni della giornata.

Il file JSON descrive invece il processo che ha condotto al risultato. Esso contiene il numero di tentativi, i punteggi di fairness, il lavoratore più penalizzato e la condizione di arresto.

Questa distinzione tra output operativo e audit log rende il sistema più verificabile. Il foglio Excel mostra il piano finale, mentre il JSON permette di comprendere come il workflow sia arrivato a quella soluzione.

7.1 Caso A

Nel Caso A l'esecuzione analizzata considera 13 lavoratori standard, identificati dalle lettere da **A** a **M**.

Il workflow ha richiesto una correzione delle preferenze e due tentativi di generazione prima di ottenere un piano valido. Successivamente è stato eseguito un tentativo di raffinamento.

La proposta prodotta dal raffinatore non ha superato la verifica hard. Il sistema ha quindi applicato correttamente il meccanismo di fallback, mantenendo il precedente **best_plan**.

I principali risultati sono riassunti nella seguente tabella.

```
{
  "dipendenti": [
    "A",
    "B",
    "C",
    "D",
    "E",
    "F",
    "G",
```

```

        "H",
        "I",
        "J",
        "K",
        "L",
        "M"
    ],
    "iterazioni_correzioni_preferenze": 1,
    "iterazioni_piano": 2,
    "iterazioni_raffinazioni": 1,
    "fairness_score": {
        "H": 5.0,
        "I": -4.0,
        "A": -26.0,
        "B": 0.0,
        "C": 0.0,
        "D": 46.0,
        "E": -12.0,
        "F": 0.0,
        "G": 10.0,
        "J": 15.0,
        "K": 0.0,
        "L": 0.0,
        "M": -45.0
    },
    "dipendente_piu_sfortunato": [
        "D"
    ],
    "condizione_di_stop": "Vincoli Hard Violati nel Raffinamen
to"
}

```

[piano_di_turni_1783275650.7322805.xlsx](#)

7.2 Caso B

Il Caso B introduce la distinzione tra lavoratori standard e specializzati.

Nell'esecuzione descritta sono presenti 19 dipendenti: 13 standard, identificati da A

a **M**, e 6 specializzati, identificati da **N** a **S**.

Questo scenario ha richiesto cinque tentativi di generazione prima di ottenere un piano valido. Il numero maggiore di iterazioni è coerente con la maggiore complessità del problema, poiché ogni turno deve essere coperto da almeno tre persone e deve includere almeno un lavoratore specializzato.

L'audit log disponibile riporta:

```
{
  "dipendenti": [
    "A",
    "B",
    "C",
    "D",
    "E",
    "F",
    "G",
    "H",
    "I",
    "J",
    "K",
    "L",
    "M",
    "N",
    "O",
    "P",
    "Q",
    "R",
    "S"
  ],
  "iterazioni_correzioni_preferenze": 0,
  "iterazioni_piano": 5,
  "iterazioni_raffinazioni": 2,
  "fairness_score": {
    "A": -22.5,
    "B": 0.0,
    "C": 10.0,
    "D": 72.0,
    "E": -20.0,
    "F": 0.0,
    "G": 45.0,
```

```

    "H": 0.0,
    "I": 13.0,
    "J": -7.5,
    "K": 10.0,
    "L": 0.0,
    "M": -3.0,
    "N": 40.0,
    "O": 25.0,
    "P": 0.0,
    "Q": -8.0,
    "R": 0.0,
    "S": 24.0
  },
  "dipendente_piu_sfortunato": [
    "D", "D"
  ],
  "condizione_di_stop": "Fairness Score Non Migliorato"
}

```

[piano_di_turni_1785227519.7006464.xlsx](#)

8. Conclusioni

Smart Scheduler affronta il problema della pianificazione dei turni attraverso un'architettura multi-agente nella quale capacità generative e strumenti simbolici svolgono compiti distinti.

I modelli linguistici vengono utilizzati nelle fasi in cui è necessario interpretare testi, riconoscere preferenze, produrre strategie, generare codice o proporre modifiche al piano.

Le componenti deterministiche vengono invece impiegate per la validazione degli schemi, la verifica dei vincoli hard, il calcolo della fairness e la selezione della soluzione migliore.

Questa separazione rappresenta il principale punto di forza del progetto. L'LLM non viene considerato una fonte infallibile, ma un componente capace di formulare proposte che devono essere controllate esternamente.

Il ciclo di estrazione, verifica e correzione riduce gli errori nella comprensione delle preferenze. Un secondo ciclo analogo viene applicato al codice CSP. La generazione del piano viene inoltre collegata a un validatore OR-Tools che può respingere le matrici non ammissibili.

Il mantenimento del `best_plan` garantisce che una proposta successiva non possa eliminare un risultato già valido. Quando il raffinamento produce un piano infeasible o incapace di migliorare la fairness, il sistema termina conservando la soluzione precedente.

La funzione di fairness aggiunge una dimensione ulteriore rispetto alla semplice fattibilità. Il sistema non si limita a chiedere se il piano sia valido, ma valuta anche quale lavoratore risulti maggiormente penalizzato.

Il criterio adottato cerca innanzitutto di ridurre la penalità massima e, in caso di parità, considera la media. Questa scelta impedisce che una buona soddisfazione complessiva nasconda una situazione particolarmente negativa per un singolo dipendente.

I risultati analizzati mostrano che il sistema riesce a produrre piani validi sia nello scenario con lavoratori omogenei sia in quello con personale specializzato.

Il Caso B richiede un numero maggiore di tentativi e produce una penalità massima più elevata. Tale risultato è coerente con la minore flessibilità introdotta dall'obbligo di includere almeno uno specializzato in ogni turno.

L'analisi del codice evidenzia comunque alcuni punti di miglioramento. Il validatore deve fissare esplicitamente anche le celle di riposo, le indisponibilità assolute devono essere trasformate in vincoli hard e alcuni campi dello schema devono essere allineati con la funzione di fairness.

È inoltre opportuno proteggere l'esecuzione del codice generato e rendere più completa l'astrazione del provider LLM.

Nel complesso, Smart Scheduler costituisce un esempio significativo di sistema neuro-simbolico. Il progetto mostra come i modelli linguistici possano essere integrati con strumenti formali senza affidare loro il controllo definitivo sulla correttezza.

Con le correzioni individuate, l'architettura può diventare una base solida per un sistema di pianificazione maggiormente verificabile, sicuro, tracciabile ed estendibile.
